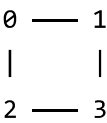


Lab 1 — Graph Representation

What is a Graph?

A **graph** models relationships between things.
Cities and roads — cities are **vertices**, roads are **edges**.

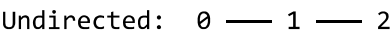
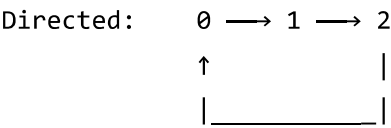


- **Vertices** (nodes) : the points $\rightarrow \{0, 1, 2, 3\}$
- **Edges** (connections) : the lines $\rightarrow \{0-1, 0-2, 1-3, 2-3\}$

Written as **G = (V, E)** where V = vertices, E = edges.

Directed vs Undirected

Type	Meaning	Example
Undirected	Edge goes both ways	$0 - 1$
Directed	Edge has one direction	$0 \rightarrow 1$



Two Ways to Store a Graph

Adjacency List

For each vertex, store only its neighbors.

```
Vertex 0 → 1, 2
Vertex 1 → 2, 3
Vertex 2 → 3
```

Adjacency Matrix

A 2D grid. `mat[i][j] = 1` means edge between `i` and `j`.

```

    0  1  2  3
0 [ 0  1  1  0 ]
1 [ 1  0  1  1 ]
2 [ 1  1  0  1 ]
3 [ 0  1  1  0 ]
```

	Adjacency List	Adjacency Matrix
Memory	$O(V + E)$	$O(V^2)$
Best for	Sparse graphs	Dense graphs

Problem 1 — Directed Graph (lab1_p1.c)

Core Idea

Store all edges as pairs (source, destination) .
To find neighbors of vertex `i` → scan all edges, print destination where source == `i` .

```
int edges[e][2];    /* edges[i][0] = source,  edges[i][1] = destination */
```

The Logic

```

for (int i = 0; i < v; i++)          /* for each vertex */
{
    printf("%d -> ", i);
    for (int j = 0; j < e; j++)      /* scan every edge */
        if (edges[j][0] == i)       /* does this edge start from i? */
            printf("%d ", edges[j][1]); /* yes → print where it goes */
    printf("\n");
}

```

Example — vertex 1 has edges 1→2 and 1→3 :

```

edges = [ [0,1], [1,2], [1,3], [2,3], [3,0] ]
          ^^^^  ^^^^
          both start from 1 → print 2 and 3

```

Output: 1 -> 2 3

Graph Theory Connection

- `edges[j][0]` = source = the vertex the arrow **leaves from**
- `edges[j][1]` = destination = the vertex the arrow **points to**
- `adjSize[i]` (out-degree) = count of edges where `edges[j][0] == i`

Sample Run

```

Enter number of vertices: 4
Enter number of edges: 5
Enter edges (source destination):
0 1
1 2
1 3
2 3
3 0

```

Adjacency List:

```

0 -> 1
1 -> 2 3
2 -> 3
3 -> 0

```

Problem 2 — Basic Graph Operations (lab1_p2.c)

Core Idea

Store the graph as an **adjacency matrix**.

Every operation (adj list, degree, properties) just reads from that matrix.

```
int mat[10][10]; /* mat[i][j] = 1 means edge i--j exists */
```

Building the Matrix

```

mat[u][w] = 1;
mat[w][u] = 1; /* undirected: set both sides */

```

If edge 2–3 is added $\rightarrow$ $\text{mat}[2][3] = 1$ AND $\text{mat}[3][2] = 1$.

Operation 1 — Print Adjacency List

Scan each row. Every column where value is 1 is a neighbor.

```
for (int j = 0; j < v; j++)
    if (mat[i][j]) printf("%d ", j);    /* 1 = neighbor, 0 = skip */
```

Operation 2 — Degree

Degree = number of edges at a vertex = count of 1s in that row.

```
int deg = 0;
for (int j = 0; j < v; j++)
    deg += mat[i][j];    /* adding 0s and 1s = count of neighbors */
```

Handshaking Lemma: Sum of all degrees = $2 \times E$

Every edge connects two vertices, so it adds 1 to two degrees.

Check: $2+2+3+2+1 = 10 = 2 \times 5 \quad \checkmark$

Operation 3 — Simple Graph

No self-loop = no vertex connected to itself = diagonal of matrix is all 0.

```
for (int i = 0; i < v; i++)
    if (mat[i][i]) { simple = 0; break; }    /* mat[i][i] = self-loop */
```

Operation 4 — Complete Graph

Every vertex connected to every other = maximum possible edges.

K3 needs 3 edges $(3 \times 2 / 2)$
 K4 needs 6 edges $(4 \times 3 / 2)$
 K5 needs 10 edges $(5 \times 4 / 2)$

```
int complete = (e == v * (v - 1) / 2);
```

Operation 5 — Connected Graph

Can you reach every vertex starting from vertex 0?

Uses **BFS** — like water spreading from one point:

Start at 0 → visit neighbors → visit their neighbors → ...

If all vertices get visited → Connected

```
int queue[10], front = 0, rear = 0;

visited[0] = 1;
queue[rear++] = 0;                                /* start from vertex 0 */

while (front < rear)
{
    int u = queue[front++];                        /* take next from queue */
    for (int i = 0; i < v; i++)
        if (mat[u][i] && !visited[i])            /* unvisited neighbor? */
        {
            visited[i] = 1;
            queue[rear++] = i;                    /* add to queue */
        }
}
```

The queue here is just a plain array. `front` moves forward as we process, `rear` moves forward as we add. Queue is empty when `front == rear` .

Sample Run

Enter number of vertices: 5

Enter number of edges: 5

Enter edges (u v):

0 1

0 2

1 2

2 3

3 4

1. Adjacency Matrix:

```
0 1 2 3 4
0 0 1 1 0 0
1 1 0 1 0 0
2 1 1 0 1 0
3 0 0 1 0 1
4 0 0 0 1 0
```

2. Adjacency List:

```
0 -> 1 2
1 -> 0 2
2 -> 0 1 3
3 -> 2 4
4 -> 3
```

3. Degree:

Vertex 0: 2

Vertex 1: 2

Vertex 2: 3

Vertex 3: 2

Vertex 4: 1

4. Properties:

Simple : YES

Complete : NO

Connected : YES

Concepts at a Glance

Concept	What it means	How code checks it
Directed edge	Arrow from u to v only	<code>edges[j][0] == i</code>
Undirected edge	Both $u \rightarrow v$ and $v \rightarrow u$	<code>mat[u][v] = mat[v][u] = 1</code>
Degree	Edges touching a vertex	Sum of row in matrix
Simple	No self-loops	<code>mat[i][i] == 0</code> for all i
Complete	Every pair connected	$E == V*(V-1)/2$
Connected	All vertices reachable	BFS visits all
Handshaking Lemma	Sum of degrees = 2E	Theoretical check

Compile and Run

```
gcc lab1_p1.c -o lab1_p1
./lab1_p1
```

```
gcc lab1_p2.c -o lab1_p2
./lab1_p2
```